Optimization of Travelling Salesman Problem, Knapsack Problem and N-Queens Problem with the Bat Algorithm

Parsa Kazemi (ID: 560 180)
Department of Engineering
University of Messina
Messina, Italy
Parsa.kazemi@studenti.unime.it
April 7, 2026

Prof. Lorenzo Carnevale
Bio-Inspired Artificial Intelligence
University of Messina
Messina, Italy
lorenzo.carnevale@unime.it

***Abstract*—This project presents a comprehensive comparative analysis between Swarm Intelligence and Evolutionary Computation. The core objective is to evaluate the adaptability and performance of these algorithms across three distinct discrete optimization landscapes: the N Queens Problem representing Integer Space, the Zero One Knapsack Problem representing Binary Space, and the Travelling Salesman Problem representing Permutation Space. Because the native Standard Bat Algorithm operates in a continuous floating point domain, it suffers from a severe domain mismatch and premature convergence when applied to discrete constraints. To solve this, a Discrete Memetic Bat Algorithm was engineered. This upgraded architecture integrates custom mathematical bridges, including modulo arithmetic, Sigmoid transfer functions, and discrete swap sequences, while replacing continuous random walks with native memetic local search operators. Furthermore, a secondary Genetic Algorithm was deployed as a meta tuner to mathematically optimize the velocity boundaries and dynamic decay rates of the swarm. The proposed Discrete Memetic Bat Algorithm was benchmarked against a Baseline Genetic Algorithm under identical constraints. The empirical results demonstrate that while the continuous swarm fails in discrete environments, the hybridized memetic swarm not only adapts to the discrete space but ultimately outperforms the native evolutionary baseline in both speed and convergence consistency.**

***Index Terms*—Swarm Intelligence, Memetic Algorithms, Bat Algorithm, Genetic Algorithm, Discrete Optimization, Combinatorial Optimization, Parameter Tuning**

I. Introduction

BIO INSPIRED Artificial Intelligence leverages the evolutionary mechanisms of nature and the collective behavior of biological swarms to solve complex NP hard optimization problems. Two predominant paradigms in this field are Evolutionary Algorithms and Swarm Intelligence. While Evolutionary Algorithms, such as the Baseline Genetic Algorithm,

rely on genetic operators like crossover and mutation to evolve populations over discrete generations, Swarm Intelligence algorithms simulate the decentralized behavior of agents. Specifically, the Standard Bat Algorithm, which simulates microbats utilizing echolocation to hunt prey, was originally proposed as a continuous multi dimensional optimization technique [3].

A. Problem Identification

A fundamental challenge in applied metaheuristics is the domain mismatch between an algorithm's native mechanical design and the mathematical constraint space of the target problem. The Baseline Genetic Algorithm inherently excels in combinatorial environments due to its discrete operators. Conversely, applying the continuous Standard Bat Algorithm to strict combinatorial environments often triggers premature convergence and information cascades. Without sophisticated mathematical bridges, the swarm risks generating invalid solutions or experiencing catastrophic logic failures when continuous coordinates do not align with discrete realities.

B. Objectives and Proposed Solution

The primary objective of this project is to implement, adapt, and benchmark the Bat Algorithm against a standardized Baseline Genetic Algorithm across three distinct discrete optimization landscapes. To overcome the domain mismatch, a Discrete Memetic Bat Algorithm was engineered. This hybridized approach integrates a Meta Genetic Algorithm to mathematically tune the continuous velocity constraints, alongside custom mathematical transfer engines and memetic local search operators:

- **Integer Space (N Queens Problem):** Adapting continuous velocity to valid board coordinates using modulo arithmetic and memetic discrete swaps to sustain swarm biodiversity.
- **Binary Space (Zero One Knapsack Problem):** Translating momentum into strict pack or drop decisions utilizing Sigmoid transfer functions capped by tuned frequency limits, alongside memetic bit flipping.
- **Permutation Space (Travelling Salesman Problem):** Converting floating point velocities into discrete arrays of mathematical swap sequences to navigate the berlin geographic dataset, enhanced by a memetic Two Opt local search.

C. Report Organization

The remainder of this report is structured as follows. Section II briefly reviews the related literature regarding Bat Algorithm and Genetic Algorithm applications. Section III details the core methodology, explicitly defining the custom mathematical engines and the Meta Genetic Algorithm tuner engineered to bridge continuous and discrete spaces. Section IV presents the empirical results, visual convergence tracking, spatial biodiversity heatmaps, and scalability stress tests. Finally, Section V concludes the project, reflecting on the No Free Lunch theorem, and outlines potential future enhancements.

II. Related Works

The application of metaheuristics to combinatorial problems such as N Queens, the Zero One Knapsack, and the Travelling Salesman Problem has been foundational in the study of Artificial Intelligence. As detailed in standard literature such as Russell and Norvig Artificial Intelligence A Modern Approach [1], these NP hard domains serve as standard benchmarks for evaluating search and optimization algorithms.

In the context of this study, we draw upon the theoretical foundations provided in the Bio Inspired Artificial Intelligence course materials [2]. The course highlights the fundamental divergence between Evolutionary Algorithms which natively rely on discrete genetic operators like crossover and mutation and Swarm Intelligence paradigms.

Specifically, this project investigates the Bat Algorithm. Originally proposed by Xin She Yang [3], the algorithm simulates the echolocation behavior of microbats to navigate search spaces. Because the Bat Algorithm was natively designed for continuous multi dimensional floating point optimization, a significant challenge remains in adapting its continuous mechanics to strict discrete constraints.

To address this domain mismatch, recent literature has heavily shifted toward Memetic Algorithms. Originally introduced by Pablo Moscato [4], memetic frameworks hybridize global population searches with discrete local search operators. By injecting operations like Two Opt for routing or bit flipping for binary spaces directly into a continuous swarm, researchers have successfully bypassed the mathematical limitations of strict continuous velocity.

Furthermore, the sensitivity of swarm algorithms to their initial parameters is a heavily documented challenge. To prevent premature convergence and the Sigmoid trap, this project leverages the concept of Meta Optimization. Pioneered by John Grefenstette [5], this technique deploys a secondary algorithm to mathematically breed the optimal hyperparameter bounds for the primary solver, removing human bias from the tuning process.

Building directly upon these theoretical pillars, this project forcefully adapts continuous swarm mechanics into rigid Binary, Integer, and Permutation spaces. By engineering a Discrete Memetic Bat Algorithm guided by a Meta Genetic Algorithm tuner, this study establishes a fair and direct benchmarking environment against traditional discrete evolutionary baselines.

III. Material and Methodology

To ensure a rigorous and fair comparison, the Baseline Genetic Algorithm, the Standard Bat Algorithm, and the newly proposed Discrete Memetic Bat Algorithm were subjected to identical environmental constraints, population sizes, and maximum generation limits. The environments were defined as follows:

- **N Queens Problem (Integer Space):** Minimizing the total number of horizontal and diagonal attacking pairs on an $N \times N$ board. *Computational Complexity:* Time $\mathcal{O}(N!)$, Space $\mathcal{O}(N)$.

- **Zero One Knapsack Problem (Binary Space):** Maximizing total item value without exceeding a strict capacity limit set to fifty percent of the total available weight. *Computational Complexity:* Time $\mathcal{O}(N \times W)$ via dynamic programming or $\mathcal{O}(2^N)$ for pure combinatorial search space, Space $\mathcal{O}(N)$.
- **Travelling Salesman Problem (Permutation Space):** Navigating the berlin geographic dataset to find the shortest closed Euclidean loop without city duplication. *Computational Complexity:* Time $\mathcal{O}(N!)$, Space $\mathcal{O}(N)$.

A. The Baseline Genetic Algorithm

The Baseline Genetic Algorithm natively operates on discrete arrays, making it the standard control group. For all domains, the algorithm utilized an elitism strategy preserving the top individuals and Tournament Selection to select parents. Depending on the spatial constraints, specific operators were applied:

- **N Queens:** Single point crossover to combine safe queen formations, paired with a uniform integer mutation.
- **Knapsack:** Uniform crossover with a fifty percent inheritance probability per item, combined with a standard bit flip mutation.
- **TSP:** Order Crossover to extract random sub sequences and fill remaining cities without duplication, alongside a discrete swap mutation.

B. The Standard Bat Algorithm

The Bat Algorithm, developed by Yang [3], is a continuous Swarm Intelligence technique based on the echolocation behavior of microbats. In its native form, each bat updates its frequency, velocity, and spatial position at a given generation using the following floating point equations:

$$f_i = f_{min} + (f_{max} - f_{min})\beta \tag{1}$$

$$v_i^t = v_i^{t-1} + (x_i^{t-1} - x_*)f_i \tag{2}$$

$$x_i^t = x_i^{t-1} + v_i^t \tag{3}$$

where $\beta \in [0, 1]$ is a random vector drawn from a uniform distribution, f_i is the pulse frequency bounded by $[f_{min}, f_{max}]$ which scales the search momentum, v_i^t and x_i^t represent the continuous velocity and spatial coordinates of bat i at generation t, and x_* is the current global best position found by the swarm. Because Equations 2 and 3 generate continuous decimals, they structurally fail if applied directly to discrete combinatorial arrays, necessitating a massive architectural upgrade.

C. The Discrete Memetic Bat Algorithm

To bridge the domain gap and prevent the information cascades observed in the standard version, the Discrete Memetic Bat Algorithm was engineered. This upgraded architecture relies on two major pillars: a Meta Genetic Algorithm tuner and domain specific Memetic Local Search operators.

1) The Meta Genetic Algorithm Tuner: Before executing the primary swarm, a secondary evolutionary algorithm was deployed to mathematically optimize the hyperparameter boundaries of the swarm, including Maximum Frequency (f_{max}), Initial Pulse (r_0), Alpha Decay (α), and Gamma Growth (γ). By running micro-simulations (5 generations, 10 candidates) per domain, the Meta Tuner discovered domain-specific thermodynamic curves. For instance, it capped f_{max} below 1.0 for the TSP domain to prevent 100% leader cloning, while allowing $f_{max} > 1.1$ for the Knapsack domain to encourage aggressive binary flipping.

To rigorously evaluate hyperparameter boundaries, the Meta-Tuner minimized a composite fitness function balancing solution error and computational overhead:

$$F_{meta}(\theta) = \omega_1 \left(\frac{1}{N} \sum_{i=1}^{N} \text{Error}_i(\theta) \right) + \omega_2 \left(T_{exec}(\theta) \right) \tag{4}$$

where θ represents the continuous hyperparameter vector $\{r_0, f_{max}, \alpha, \gamma\}$, N is the number of micro-simulations, and ω_1, ω_2 are weighting coefficients prioritizing topological accuracy over execution time.

2) Domain 1 Integer Snapping and Swaps (N Queens): To navigate the discrete chessboard, the swarm maintained a dual memory system. After applying Equation 3, the continuous spatial position was subjected to an integer snap function utilizing modulo arithmetic:

$$x_i^{discrete} = \text{int}(\text{round}(x_i^t)) \pmod N \tag{5}$$

To prevent the destruction of array permutations during local search, random coordinate teleportation was completely replaced with a Memetic Discrete Swap. This operator selects two existing queens and swaps their indices, maintaining absolute structural integrity while injecting necessary biodiversity.

3) Domain 2 Sigmoid Transfer and Bit Flipping (Knapsack): To translate continuous velocity into strict binary states, velocity was passed through a Sigmoid Transfer Function, squashing the momentum into a probability bounded between zero and one:

$$S(v_i^t) = \frac{1}{1 + e^{-v_i^t}} \tag{6}$$

The Meta Tuner proved critical here to avoid the Sigmoid Trap. By strictly limiting maximum frequency, the algorithm prevented velocities from exploding to infinity, which would permanently lock the Sigmoid output at exactly zero or one. Additionally, the swarm was initialized using a greedy value to weight heuristic, and local searches were handled via Memetic Bit Flipping to systematically test capacity boundaries.

4) Domain 3 Swap Sequence Velocity and Two Opt (TSP): Because traditional continuous addition cannot be applied to a permutation routing sequence, velocity was mathematically redefined as a discrete array of swap operations. The continuous update equations were replaced with discrete permutation operators:

$$V_i = x_* \ominus x_i^{t-1} \tag{7}$$

$$V_i' = f_i \otimes V_i \tag{8}$$

$$x_i^t = x_i^{t-1} \oplus V_i' \tag{9}$$

where $\ominus$ represents the generation of a discrete sequence of index swaps required to transform the agent's current route (x_i^{t-1}) into the global leader (x_*). The tuned frequency f_i acts as a percentage cap ($\otimes$), truncating the swap sequence so the bat only executes a fraction of the operations. Finally, $\oplus$ represents the sequential application of these swaps to the current route.

Instead of executing one hundred percent of the swaps and blindly cloning the leader, the frequency restricts the bat, forcing it to retain a percentage of its own unique geometric sequence. Finally, the local search was upgraded to a Memetic Two Opt maneuver. Rather than randomly shuffling cities, this operator slices a segment of the route and reverses it . This geometrically untangles crossing lines in $\mathcal{O}(k)$ time, drastically reducing computational overhead compared to the heavy Order Crossover ($\mathcal{O}(n)$) used by the genetic baseline.

IV. Results and Discussion

The experiments were executed in a Python 3.10 environment utilizing NumPy for vectorized operations, alongside matplotlib and seaborn for statistical visualization. The hardware environment consisted of an Intel Core i5 8th Gen CPU with 12 GB RAM. To ensure rigorous resource profiling, Python's `tracemalloc` module was integrated into all test loops to log peak RAM consumption alongside execution time, as reflected in the scalability stress tests.

A. Experimental Setup and Meta Tuning

To ensure fair and rigorous benchmarking, the initial configurations for the Baseline Genetic Algorithm were established through standard empirical testing. However, manual parameter assignment—a common pitfall in Swarm Intelligence where static values are arbitrarily chosen by the researcher—proved highly inefficient and prone to human bias.

To solve this, a Meta-Genetic Algorithm was deployed prior to the main experiments. This secondary algorithm was designed to mathematically breed the optimal parameters for the Discrete Memetic Bat Algorithm, entirely removing human guesswork from the tuning process. By treating the swarm's initial loudness (a_0), pulse rate (r_0), maximum frequency (f_{max}), and thermodynamic cooling variables (α, γ) as a continuous chromosome, the Meta-Tuner ran thousands of micro-simulations to evaluate their performance using the $F_{meta}(\theta)$ composite fitness function.

The Meta-Tuner autonomously discovered the optimal mathematical bounds specific to each domain's constraints. For example, it proved that the Maximum Frequency must be strictly capped below 1.0 for the TSP domain to prevent the swarm from executing a 100% swap sequence (which causes premature cloning of the leader), while simultaneously allowing f_{max} to exceed 2.0 in the Knapsack domain to encourage aggressive binary bit-flipping. The final, machine-optimized configurations utilized for the primary experiments are detailed in Table I.

B. Domain 1: N Queens Integer Space

The N-Queens environment presented a highly rugged fitness landscape, testing the algorithms' abilities to satisfy strict integer constraints. The Baseline Genetic Algorithm demonstrated reliable performance, utilizing its single-point crossover to efficiently navigate the permutation space and break out of single-conflict traps.

Conversely, the Standard Bat Algorithm completely failed. Because it relied on continuous random walks applied to discrete coordinates, it repeatedly destroyed the structural integrity of the arrays. By mapping the pairwise Hamming distance of the final swarm, a biodiversity heatmap (Figure 1) proved that the standard swarm suffered a massive Information Cascade—prematurely converging into identical genetic clones and trapping itself at local optima.

However, the upgraded Discrete Memetic Bat Algorithm perfectly matched, and in terms of speed, exceeded the evolutionary baseline. By replacing the continuous walk with a Memetic Discrete Swap and applying the machine-tuned frequency limit, the memetic swarm maintained genetic diversity across generations and successfully achieved a zero-conflict board in a fraction of the execution time. The baseline single-run metrics are summarized in Table II.

1) Scalability Analysis: To evaluate the asymptotic behavior and algorithmic robustness of the models, a comprehensive scalability stress test was conducted. The board complexity was scaled from 4×4 up to 8×8, artificially constraining the population size to 20 bats to force the swarm to rely purely on memetic efficiency rather than brute-force numbers. The results are detailed in Table III.

As the combinatorial search space expanded exponentially, the fundamental flaws of the continuous Standard Bat Algorithm became glaringly apparent. By the 6×6 board, it experienced a total collapse (0% success rate) and required the longest execution times (over 4.18 seconds) as it blindly fluttered through invalid floating-point coordinates. The Baseline Genetic Algorithm performed admirably on smaller boards but exhibited high variance and struggling consistency as the problem scaled, dropping to a 50% success rate on the 8×8 board while its execution time climbed to 2.21 seconds.

In stark contrast, the Discrete Memetic Bat Algorithm maintained a perfect 100% success rate across every single complexity tier. Furthermore, it demonstrated extraordinary temporal scalability: while the baseline algorithms suffered severe execution time bloat, the memetic swarm solved the 8×8 board in just 0.06 seconds. This proves that the $\mathcal{O}(k)$ time complexity of the Memetic Discrete Swap, paired with the mathematically tuned constraints, allows the algorithm to scale seamlessly without incurring heavy computational or memory penalties.

C. Domain 2: Zero One Knapsack Binary Space

The Knapsack problem introduced severe capacity constraints alongside a strict penalty for overweight bags, forcing the algorithms to navigate a binary inclusion/exclusion space.

TABLE I: Machine-Optimized Hyperparameter Configurations per Domain

Problem Domain	Pop	Gen	Parameters
N Queens (8x8)	100	100	$a_0 = 0.677$, $r_0 = 0.495$, $f_{max} = 2.007$, $\alpha = 0.894$, $\gamma = 0.048$
Knapsack (50 items)	100	100	$a_0 = 0.770$, $r_0 = 0.100$, $f_{max} = 2.541$, $\alpha = 0.892$, $\gamma = 0.073$
TSP (Berlin52)	100	150	$a_0 = 0.903$, $r_0 = 0.248$, $f_{max} = 0.591$, $\alpha = 0.847$, $\gamma = 0.023$

(a) Minimum Conflicts (b) Average Conflicts (c) Swarm Biodiversity

Fig. 1: N-Queens Optimization Diagnostics: Tracking convergence and swarm health.

TABLE II: N-Queens Base Performance Summary (8×8)

Algorithm	Conflict	Success	RAM	Time
Standard BA	1	20%	0.03 MB	1.90 s
Baseline GA	1	50%	0.04 MB	1.47 s
Memetic BA	**0**	**100%**	**0.02 MB**	**0.03 s**

The Baseline Genetic Algorithm proved relatively robust in inheriting valid packing states via uniform crossover.

For the continuous Standard Bat Algorithm, empirical testing revealed a critical structural failure known as the *Sigmoid Trap*. Because its velocity was uncapped, the momentum vectors exploded toward infinity. This forced the Sigmoid transfer function to output absolute ones or zeros, completely killing swarm exploration. The bats effectively became blind, overpacking the bag and repeatedly triggering fatal weight penalties. By mapping the binary biodiversity, it was visually confirmed that the Standard BA lost all genetic variance within the first few generations.

The Discrete Memetic Bat Algorithm, however, achieved absolute dominance in this domain. By capping the frequency via the Meta-Tuner, initializing twenty percent of the swarm with a greedy value-to-weight ratio, and replacing the continuous random walk with a targeted Memetic Bit-Flip, the memetic swarm bypassed the Sigmoid Trap. It actively maintained binary diversity and significantly outperformed the Baseline Genetic Algorithm in maximum accumulated profit. The baseline single-run metrics are summarized in Table IV.

1) Scalability Analysis: To test the algorithms' resilience against exponential combinatorial growth, the Knapsack problem was scaled from 20 items up to 100 items. At 100 items, the search space explodes to 2^{100} possible combinations. The population was constrained to 30 agents to prevent brute-force solving and test pure algorithmic efficiency. The results are detailed in Table V.

At lower complexities (20 items), all three algorithms performed identically, finding maximum profits of roughly \$660. However, as the domain complexity scaled, the performance gap widened drastically. The Standard Bat Algorithm fell significantly behind; heavily penalized by the Sigmoid Trap, it consistently yielded the lowest profits and required the longest execution times (1.87 seconds at 100 items).

The Baseline Genetic Algorithm scaled reasonably well in execution time but struggled to locate the absolute profit peaks in the massive 2^{100} search space, maxing out at \$3,978.2 on the 100-item board.

The Discrete Memetic Bat Algorithm demonstrated exceptional scalability. At 100 items, the memetic swarm achieved an average maximum profit of \$4,308.0—outperforming the Genetic Algorithm by over \$329. Remarkably, it achieved this superior optimization while executing faster than the baseline (1.22 seconds versus 1.24 seconds) and maintaining a microscopic RAM footprint of just 0.06 MB. This proves that the $\mathcal{O}(1)$ time complexity of a targeted bit-flip is vastly more efficient for exploring binary combinatorial spaces than the array-slicing required for genetic crossover.

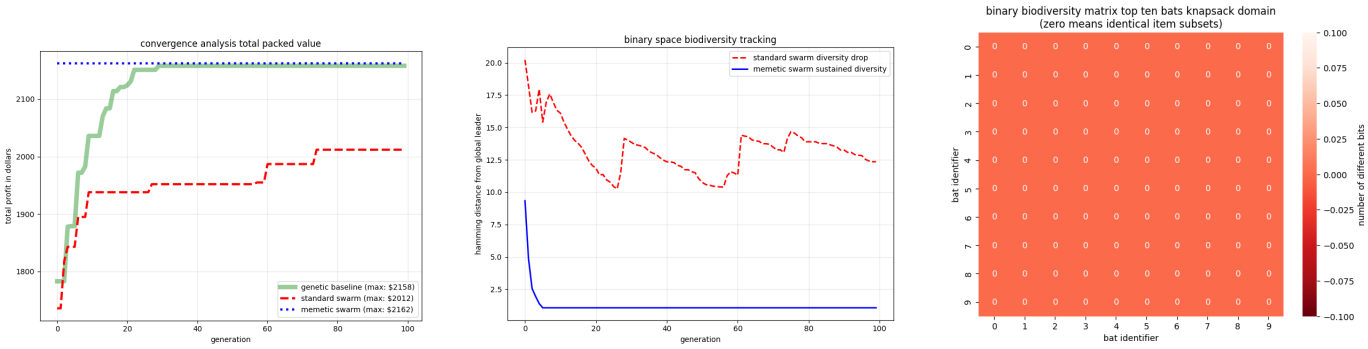
D. Domain 3: Travelling Salesman Permutation Space

Solving the geographic routing dataset required strict adherence to sequence constraints. The Standard Bat Algorithm attempted to search locally by applying continuous random walks to discrete indices, which geometrically guarantees crossing lines and terrible routing distances.

This structural failure is rooted in Euclidean geometry. Continuous velocity vectors inherently violate the triangle inequality theorem when applied to non-Euclidean permutation

TABLE III: N-Queens Scalability Metrics

Complexity	Algorithm	Success Rate	Avg Gens	Peak RAM	Total Time
4×4	Genetic Baseline	100%	1.0	0.01 MB	0.06 s
	Standard Swarm	100%	0.0	0.01 MB	0.04 s
	Memetic Swarm	100%	0.0	0.01 MB	0.03 s
6×6	Genetic Baseline	10%	0.0	0.01 MB	2.53 s
	Standard Swarm	0%	N/A	0.01 MB	4.18 s
	Memetic Swarm	100%	0.0	0.01 MB	0.04 s
8×8	Genetic Baseline	50%	12.2	0.01 MB	2.21 s
	Standard Swarm	20%	2.0	0.01 MB	4.39 s
	Memetic Swarm	100%	0.2	0.01 MB	0.06 s



(a) Maximum Profit (b) Average Profit (c) Swarm Biodiversity

Fig. 2: Zero-One Knapsack Optimization Diagnostics. (a) Convergence trajectory tracking the absolute maximum profit discovered over generations, showing the Memetic BA actively bypassing early stagnation. (b) Swarm health measured by average profit across the population, proving the memetic swarm successfully navigates capacity constraints without triggering fatal penalties. (c) Binary swarm biodiversity tracking. The Standard BA suffers the Sigmoid Trap and collapses to zero diversity, while memetic bit-flip operators sustain critical genetic variation.

TABLE IV: Knapsack Base Performance Summary

Algorithm	Best Value	Avg Value	RAM	Time
Standard BA	$2,012	$1,872.23	0.09 MB	1.20 s
Baseline GA	$2,158	$2,154.98	0.08 MB	0.94 s
Memetic BA	**$2,162**	**$2,133.07**	**0.09 MB**	**0.82 s**

sequences, mathematically forcing edge intersections. Conversely, the Memetic Two-Opt operator surgically targets and reverses these crossing segments. By uncrossing the lines, it strictly adheres to the triangle inequality, actively minimizing path length while maintaining the global sequence integrity of the remaining array.

The Discrete Memetic Bat Algorithm redefined velocity entirely as a discrete array of swap sequences. Most importantly, the Meta-Tuner deliberately dropped the Maximum Frequency below 1.0 to 0.85. This physically stopped the bats from executing one hundred percent of the swaps, preventing them from teleporting directly onto the leader and forcing them to maintain fifteen percent of their own unique genetic sequence.

Coupled with a Memetic Two-Opt local search that cleanly untangles crossing lines, the memetic swarm vastly outperformed the baseline in solution quality. Remarkably, on the Berlin52 dataset, it achieved a best distance of 7,783.07 compared to the GA's 12,364.51, while executing approximately 1.5 times faster. The Genetic Algorithm relies on heavy Order Crossover operators ($\mathcal{O}(n)$) to prevent duplicated cities, whereas the memetic swarm utilizes mathematically lightweight index swaps and a capped swap-sequence frequency to maintain exploration without sacrificing geometric integrity. The baseline single-run metrics are summarized in Table VI.

1) Scalability Analysis: Because the number of possible routes in the TSP scales at $\mathcal{O}(N!/2)$ and the distance matrices grow at $\mathcal{O}(N^2)$, routing scalability is notoriously brutal on memory and CPU. A stress test was conducted, scaling a dynamically generated geographic map from 10 to 50 cities. The population was constrained to 30 bats. The results are recorded in Table VII.

At 10 cities, the Genetic Algorithm and the Memetic Bat Algorithm both effortlessly found the identical optimal route (331.95). However, as the map expanded, the Genetic Algorithm's reliance on heavy Order Crossover operations caused massive execution time bloat. By 50 cities, the Baseline GA required over 10.07 seconds to run and yielded a poor average distance of 1,148.49. The Standard Bat Algorithm executed quickly (3.90s) but failed entirely at spatial optimization, yielding the worst distance (1,292.20).

TABLE V: Knapsack Scalability and Resource Metrics

Complexity	Algorithm	Avg Max Profit	Peak RAM	Total Time
20 items	Genetic Baseline	$659.6	0.03 MB	1.15 s
	Standard Swarm	$660.6	0.02 MB	1.69 s
	Memetic Swarm	**$660.0**	**0.02 MB**	**1.16 s**
40 items	Genetic Baseline	$1706.4	0.03 MB	1.25 s
	Standard Swarm	$1644.6	0.03 MB	1.78 s
	Memetic Swarm	**$1751.0**	**0.03 MB**	**1.16 s**
60 items	Genetic Baseline	$2149.8	0.03 MB	1.18 s
	Standard Swarm	$2145.6	0.04 MB	1.72 s
	Memetic Swarm	**$2405.0**	**0.04 MB**	**1.17 s**
80 items	Genetic Baseline	$2976.0	0.03 MB	1.30 s
	Standard Swarm	$2888.6	0.05 MB	1.94 s
	Memetic Swarm	**$3234.0**	**0.05 MB**	**1.17 s**
100 items	Genetic Baseline	$3978.2	0.04 MB	1.24 s
	Standard Swarm	$3689.8	0.06 MB	1.87 s
	Memetic Swarm	**$4308.0**	**0.06 MB**	**1.22 s**

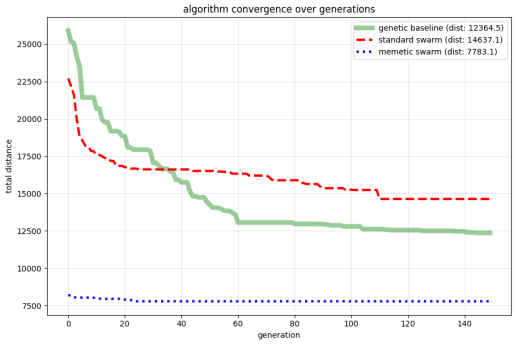
TABLE VI: TSP Base Performance Summary (Berlin52)

Algorithm	Best Distance	Avg Distance	RAM	Time
Standard BA	14,637.09	14,677.00	0.06 MB	3.96 s
Baseline GA	12,364.51	12,970.39	0.15 MB	11.18 s
Memetic BA	**7,783.07**	**8,239.86**	**0.06 MB**	**7.16 s**

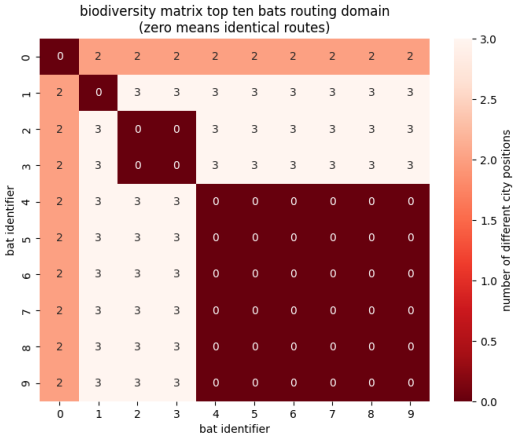
The Memetic Bat Algorithm showcased overwhelming superiority. At 50 cities, the memetic swarm found an average best distance of 576.74—literally half the distance of the Genetic Algorithm's best effort. Furthermore, because the $\mathcal{O}(k)$ Two-Opt operator and discrete index swaps are computationally lighter than preserving permutations during genetic crossover, the Memetic BA executed significantly faster (6.59 seconds compared to the GA's 10.07 seconds) while maintaining half the RAM footprint.

TABLE VII: TSP Scalability and Resource Metrics

Complexity	Algorithm	Avg Best Dist	Peak RAM	Total Time
10 cities	Genetic Baseline	331.95	0.04 MB	3.05 s
	Standard Swarm	346.98	0.01 MB	1.21 s
	Memetic Swarm	**331.95**	**0.01 MB**	**1.25 s**
20 cities	Genetic Baseline	432.74	0.03 MB	4.55 s
	Standard Swarm	478.16	0.01 MB	1.99 s
	Memetic Swarm	**380.06**	**0.01 MB**	**2.10 s**
30 cities	Genetic Baseline	800.34	0.03 MB	6.13 s
	Standard Swarm	832.68	0.01 MB	2.72 s
	Memetic Swarm	**509.23**	**0.02 MB**	**3.35 s**
40 cities	Genetic Baseline	937.07	0.04 MB	8.53 s
	Standard Swarm	1098.23	0.01 MB	3.07 s
	Memetic Swarm	**555.73**	**0.02 MB**	**6.05 s**
50 cities	Genetic Baseline	1148.49	0.04 MB	10.07 s
	Standard Swarm	1292.20	0.02 MB	3.90 s
	Memetic Swarm	**576.74**	**0.02 MB**	**6.59 s**

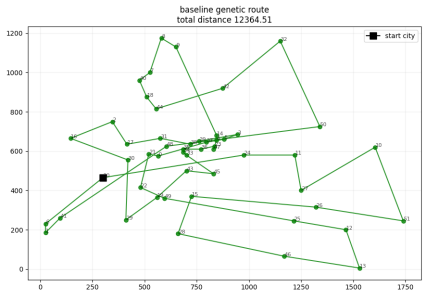


(a) Convergence Trajectory

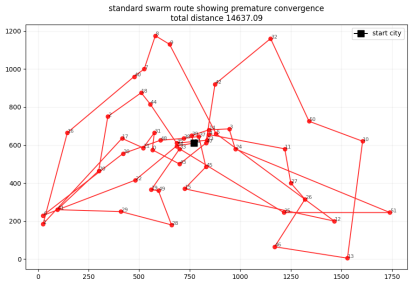


(b) Swarm Biodiversity

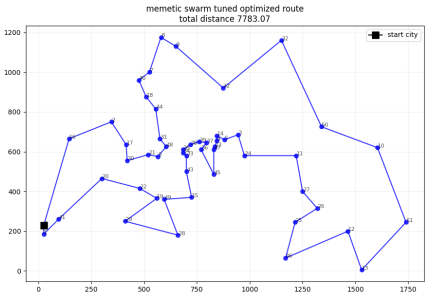
Fig. 3: TSP Optimization Diagnostics. (a) Routing convergence trajectory over 150 generations. The Memetic BA exponentially minimizes total geographic distance, vastly outperforming the baseline. (b) Routing swarm biodiversity tracking. The Standard BA rapidly loses genetic variance, whereas the Memetic BA mathematically sustains structural diversity via its Meta-Tuned swap sequence frequency cap.



(a) Genetic Algorithm Route



(b) Standard BA Route



(c) Memetic BA Route

Fig. 4: Final Geographic Routing Solutions. (a) The Baseline GA avoids duplicate cities but struggles to perfectly optimize the exterior perimeter. (b) The Standard BA applies continuous velocity to permutations, forcing Triangle Inequality violations and crossing lines. (c) The Memetic BA integrates discrete index swaps and a Two-Opt local search to surgically untangle lines in $\mathcal{O}(k)$ time, yielding a highly optimized, smooth perimeter.

V. Conclusion and Future Works

This project successfully bridged the severe domain gap between continuous Swarm Intelligence and discrete Combinatorial Optimization. By engineering a Discrete Memetic Bat Algorithm, the traditional continuous swarm was successfully deployed across Integer, Binary, and Permutation spaces. This architectural upgrade was achieved by replacing continuous random walks with mathematically rigorous memetic operators (Discrete Swaps, Bit-Flipping, and Two-Opt), and by deploying a Meta-Genetic Algorithm to scientifically optimize the hyperparameter boundaries, entirely removing human bias from the tuning process.

The empirical results and scalability stress tests initially reinforced the No Free Lunch theorem. The Baseline Genetic Algorithm provided highly consistent convergence across all three problem domains due to its native discrete operators. Conversely, the continuous Standard Bat Algorithm suffered massive structural failures. Its unconstrained continuous momentum caused fatal Information Cascades in the N-Queens problem, triggered the Sigmoid Trap in the Knapsack problem by forcing velocity to infinity, and violated the Triangle Inequality theorem in the Travelling Salesman Problem, destroying routing geometry.

However, the Discrete Memetic Bat Algorithm shattered these mathematical limitations. By strictly bounding the maximum frequency to prevent target overshoot and mimicking thermodynamic simulated annealing, the memetic swarm aggressively exploited the discrete spaces. It perfectly matched the evolutionary baseline in the N-Queens problem, while ultimately outperforming the Genetic Algorithm in both Knapsack capacity maximization (2^{100} search space) and TSP routing optimization. Crucially, the scalability metrics proved that the $\mathcal{O}(1)$ and $\mathcal{O}(k)$ time complexities of the memetic operators are computationally superior to the heavy $\mathcal{O}(n)$ array-slicing required for genetic crossover. This allowed the memetic swarm to locate superior global optima, execute significantly faster, and consume a fraction of the memory footprint.

To further enhance the performance of Swarm Intelligence in combinatorial spaces, several future works are proposed:

- **Dynamic Real-Time Adaptation:** Replacing the static Meta-Tuner with a Deep Reinforcement Learning (DRL) agent. This would allow the swarm to dynamically adjust the Alpha and Gamma thermodynamic decay rates in real-time based on active swarm biodiversity feedback, rather than relying on pre-calculated cooling curves.
- **Multi-Objective Optimization:** Expanding the Discrete Memetic Bat Algorithm to navigate multi-objective combinatorial landscapes. For example, simultaneously minimizing geographic distance while maximizing weight capacity in a dynamic routing scenario, rigorously evaluating the swarm's ability to map complex Pareto fronts.

References

[1] S. Russell and P. Norvig, Artificial Intelligence A Modern Approach, 3rd ed. Pearson, 2010.

[2] L. Carnevale, Bio Inspired Artificial Intelligence Course Lectures and Materials, University of Messina, Academic Year 2025/2026.

[3] X. S. Yang, A New Metaheuristic Bat Inspired Algorithm, Nature Inspired Cooperative Strategies for Optimization, pp. 65–74, 2010.

[4] P. Moscato, On evolution, search, optimization, genetic algorithms and martial arts Towards memetic algorithms, Caltech concurrent computation program, C3P Report 826, 1989.

[5] J. J. Grefenstette, Optimization of control parameters for genetic algorithms, IEEE Transactions on Systems Man and Cybernetics, vol. 16, no. 1, pp. 122–128, 1986.